```
model = torch.nn.Sequential(
    torch.nn.Linear(10, 150),
    torch.nn.ReLU(),
    torch.nn.Linear(150, 4),
    torch.nn.ReLU(),
)

loss_fn = torch.nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

We've set up a two-layer model with `ReLU` (rectified linear units) activation functions, defined a mean-squared error loss function, and set up an optimizer. All we have to do to train this model, given that we have some labeled training data, is start a training loop:

```
for step in range(100):
    y_pred = model(x)
    loss = loss_fn(y_pred, y_correct)
       optimizer.zero_grad()
       loss.backward()
       optimizer.step()
```

The `x` variable is the input data to the model. The `y_correct` variable is a tensor representing the labeled, correct output. We make the prediction using the model, calculate the loss, and then compute the gradients using the `backward()` method on the last node in the computational graph (which is almost always the `loss` function). Then we just run the `step()` method on the optimizer, and it will run a single step of gradient descent. If we need to build more complex neural network architectures than the sequential model, we can write our own Python class, inhereit from PyTorch's module class, and use that instead:

```
from torch.nn import Module, Linear

class MyNet(Module):
   def __init__(self):
       super(MyNet, self).__init__()
       self.fc1 = Linear(784, 50)
       self.fc2 = Linear(50, 10)

   def forward(self, x):
       x = F.relu(self.fc1(x))
       x = F.relu(self.fc2(x))
       return x
```